LABORATORY EXPERIMENT REPORT

EXPERIMENT 6

END-TO-END ML PIPELINE IMPLEMENTATION

STUDENT DETAILS

Name

Y.V.R. Kasyap

Registration Number

231FA04E47

Section: 8

Date: 16-10-2026

PUBLIC GITHUB REPOSITORY

https://github.com/kasyap0420/Experiment-6-MLOps

Aim

To convert a machine learning workflow into a reusable, executable and reproducible end-to-end ML pipeline covering data preparation, preprocessing, feature engineering, model training, evaluation and prediction.

Task 1 - Dataset Preparation

Dataset: Breast Cancer Wisconsin (Diagnostic), loaded through scikit-learn; 569 records and 30 numerical input features. Target classes: 0 = malignant, 1 = benign. Class distribution: {'1': 357, '0': 212}.

Data validation and cleaning: Missing feature values before cleaning: 0; duplicate rows removed: 0; retained samples: 569. No categorical encoding was required for this dataset.

Split: 80% training and 20% testing; stratification enabled; random_state = 42.

Task 2 - Design of the End-to-End Pipeline

Workflow: Data loading to validation/cleaning to feature engineering to model training to evaluation to prediction. The trained model and transformations are stored together for consistent inference.

Feature engineering: Cast all features to float32. The fitted SimpleImputer and StandardScaler are inside the scikit-learn Pipeline, so they learn parameters only from training data.

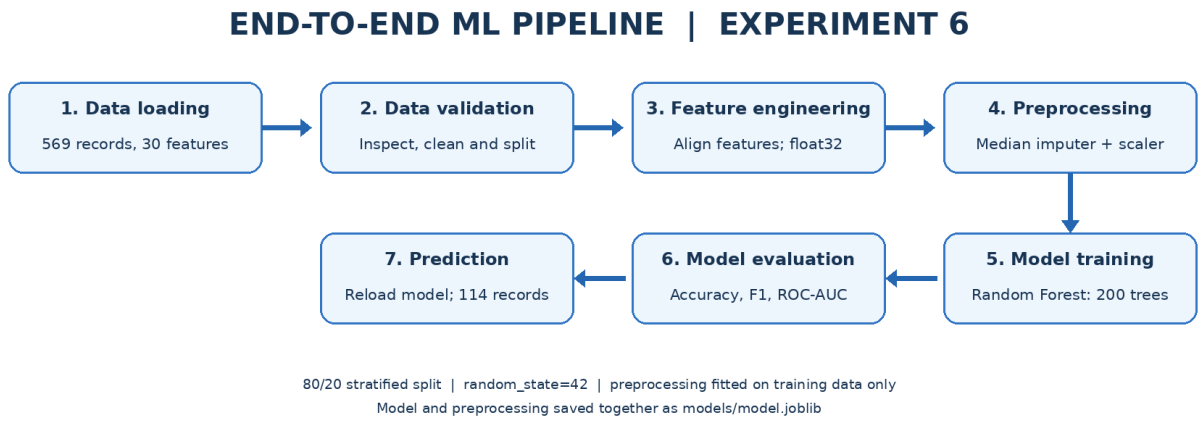


Figure 1. End-to-end ML pipeline architecture

Stage inputs and outputs:

Stage	Input	Output
Data loading	scikit-learn dataset	Features X and labels y
Cleaning	Raw X, y	Cleaned dataset, quality report
Train/test split	Cleaned X, y	Training and held-out test sets
Feature engineering	Split feature sets	Aligned numerical features
Training	Train features, labels, settings	Fitted and saved Pipeline
Evaluation	Fitted Pipeline and test set	Metrics JSON
Prediction	Saved Pipeline and test features	Prediction CSV

Task 3 - Modular Pipeline Implementation

The experiment is organized into independent, reusable Python modules:

Module	Purpose
data_loader.py	Load dataset; inspect features, target and dataset quality
preprocessing.py	Clean dataset and perform train/test split
feature_engineering.py	Align features, convert numeric types, save transformation summary
train.py	Build and train the saved scikit-learn Pipeline
evaluate.py	Calculate classification metrics and compare repeated runs
predict.py	Reload saved model; generate and export predictions
main.py	Orchestrate all stages and the reproducibility check

Configuration: config/config.yaml; model: RandomForestClassifier (200 estimators; random_state = 42); imputation: median; scaling: standard. The model and fitted preprocessing pipeline are stored in models/model.joblib.

```
12
13  def build_pipeline(n_estimators: int = 200, random_state: int = 42,
14                     imputer_strategy: str = "median") -> Pipeline:
15      """Build the scikit-learn Pipeline: Imputer -> Scaler -> RandomForest."""
16      pipeline = Pipeline([
17          ("imputer", SimpleImputer(strategy=imputer_strategy)),
18          ("scaler", StandardScaler()),
19          ("model", RandomForestClassifier(n_estimators=n_estimators, random_state=random_state)),
20      ])
21      return pipeline
22
23
```

Figure 2. Genuine VS Code screenshot of the train.py Pipeline implementation

Task 4 - Execution and Model Evaluation

The full workflow is executed with **python src/main.py** from the exp-6 directory. Saved results from two runs are available in outputs/. The figure below is the supplied terminal execution screenshot.

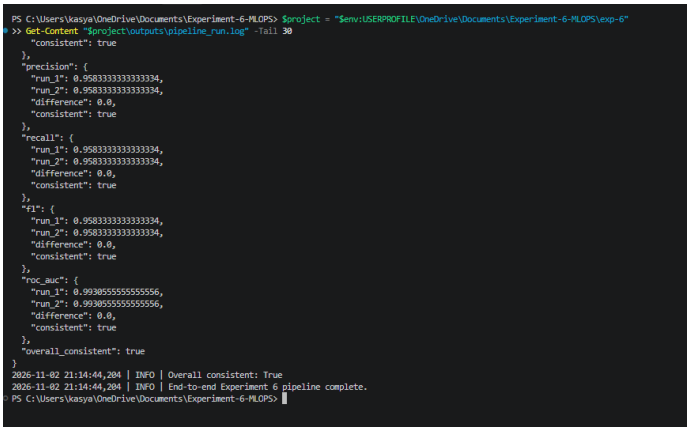


Figure 3. Genuine terminal execution evidence

Metric	Run 1 value	Meaning
Accuracy	0.9474	Overall proportion classified correctly
Precision	0.9583	Precision for benign (class 1)
Recall	0.9583	Recall for benign (class 1)
F1-score	0.9583	F1 for benign (class 1)
ROC-AUC	0.9931	Discrimination using probability of class 1

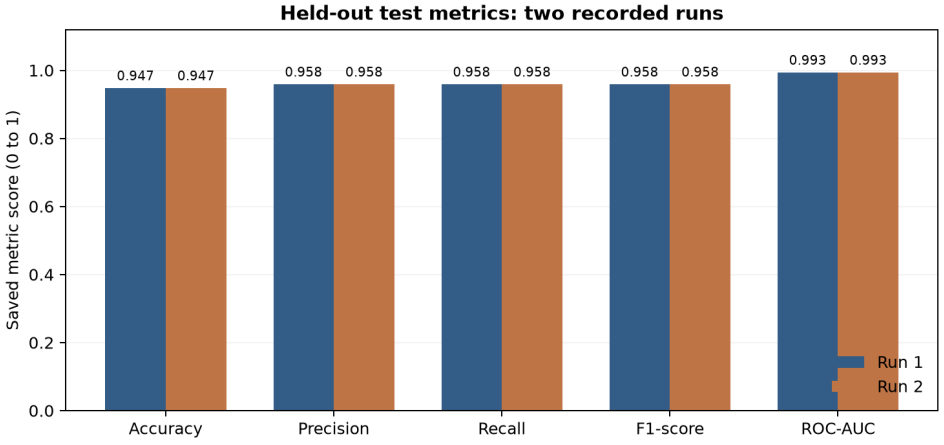


Figure 4. Recorded evaluation metrics for Runs 1 and 2

Task 5 - Reproducibility and Prediction

Two runs used the saved dataset and configuration with random_state = 42. Metric consistency: **Passed**. Prediction class and confidence consistency: **Passed**.

Metric	Run 1	Run 2	Absolute difference	Match
Accuracy	0.947368	0.947368	0.000000	Yes
Precision	0.958333	0.958333	0.000000	Yes
Recall	0.958333	0.958333	0.000000	Yes
F1-score	0.958333	0.958333	0.000000	Yes
ROC-AUC	0.993056	0.993056	0.000000	Yes

Prediction: 114 held-out test records were predicted using the reloaded saved pipeline. Outputs contain record IDs, predicted classes, labels, confidences, timestamps and model versions.

Record ID	Class	Label	Confidence	Model version
REC0001	0	malignant	1.0	v1.0
REC0002	1	benign	1.0	v1.0

Task 6 - Documentation, Observations and Result

Public GitHub repository: https://github.com/kasyap0420/Experiment-6-MLOps.

Project structure: exp-6/config/ (settings); exp-6/src/ (pipeline modules); exp-6/models/ (saved pipeline); exp-6/outputs/ (run metrics, CSV predictions and comparison); exp-6/report_assets/ (diagram, chart and terminal screenshot); exp-6/generate_report.py (PDF generation).

Observations: 569 records and 30 input features were processed. Both runs saved 114 predictions. All five evaluation metrics matched between the saved runs. The report generator verifies required files and checks the saved comparison before issuing this report.

Execution log: outputs/pipeline_run.log is available for inspection of runtime messages and any encountered errors.

Actual Issues Encountered and Corrective Actions

The verified final ML runs completed successfully. The following issues occurred during project setup and report preparation, not as unverified model-training failures:

Observed issue	Corrective action
ReportLab was missing when the maintenance dependency check ran.	Installed ReportLab in the Python environment; report generation then succeeded.
The report could not finalize when the required genuine terminal screenshot was absent.	Captured and saved the actual terminal screenshot in report_assets.
VS Code reported eight unresolved ReportLab imports while Python 3.12 was selected.	Selected the installed Python 3.14 interpreter; the eight warnings disappeared.
An earlier report split the evaluation-metrics table across two pages.	Updated ReportLab pagination to keep all five metric rows together.

Result: The reusable end-to-end ML pipeline covers dataset loading, cleaning, feature engineering, training, evaluation, saved-model inference and documented reproducibility checks.

Evidence Figures - Dataset and Inference

The following figures were constructed exclusively from the saved experiment JSON/CSV outputs and fitted Random Forest model. Original data source filenames and SHA-256 fingerprints are recorded in **report_assets/evidence_manifest.json**. No fabricated values or simulated screenshots have been used.

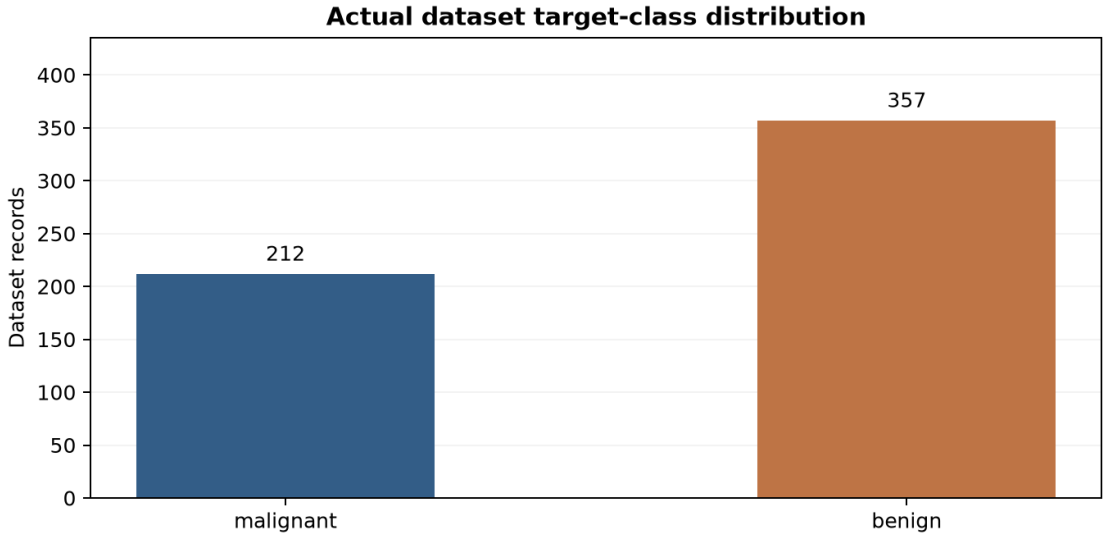


Figure 5. Actual target-class distribution across all original dataset records

Figure 5 uses outputs/run_1_dataset_inspection.json; these are actual dataset target labels, not model predictions.

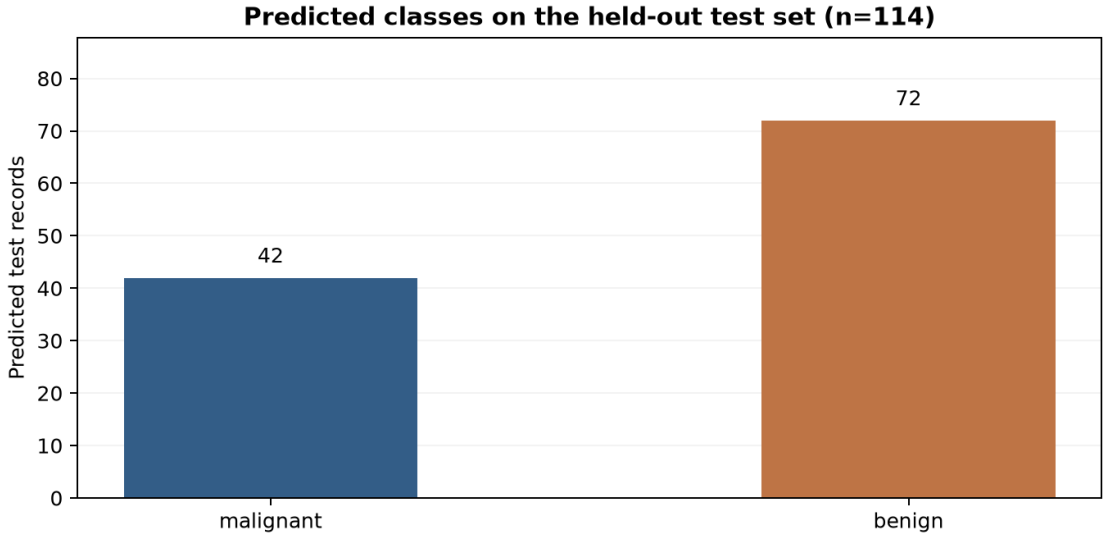


Figure 6. Predicted classes for the held-out test records

Figure 6 uses outputs/run_1_predictions.csv. Predicted-class totals are not the same as ground-truth class totals or classification correctness.

Evidence Figures - Confidence and Features

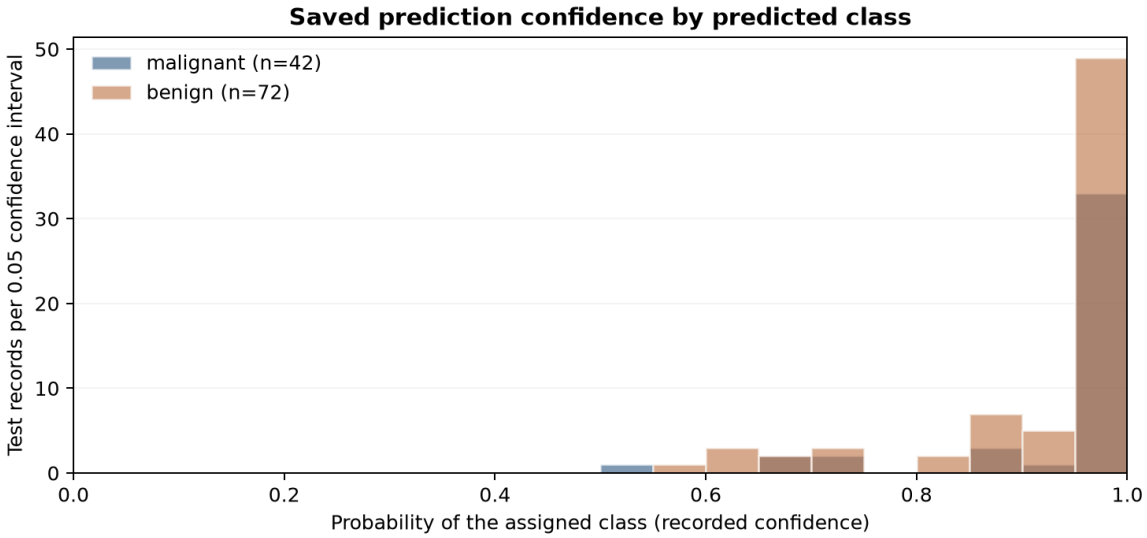


Figure 7. Distribution of saved prediction confidence scores

Each bar represents an interval of confidence values read from the saved prediction CSV, grouped by the predicted class. Confidence is not a measure of correctness.

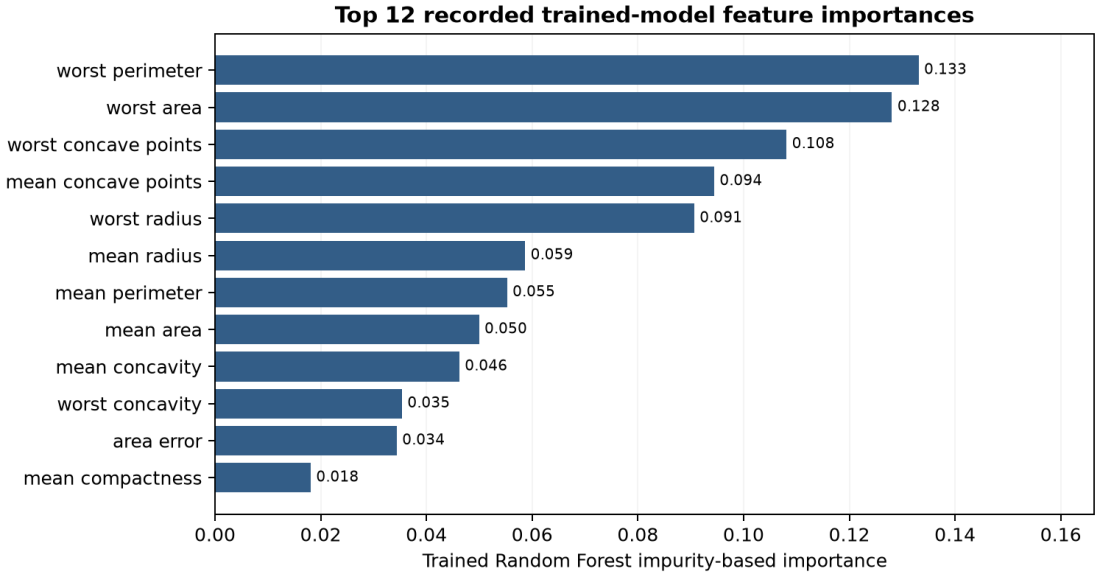


Figure 8. Top 12 impurity-based feature importances from the fitted model

These feature importances are extracted from the trained Random Forest inside models/model.joblib. They do not establish causal relationships.

Evidence Figures - Reproducibility

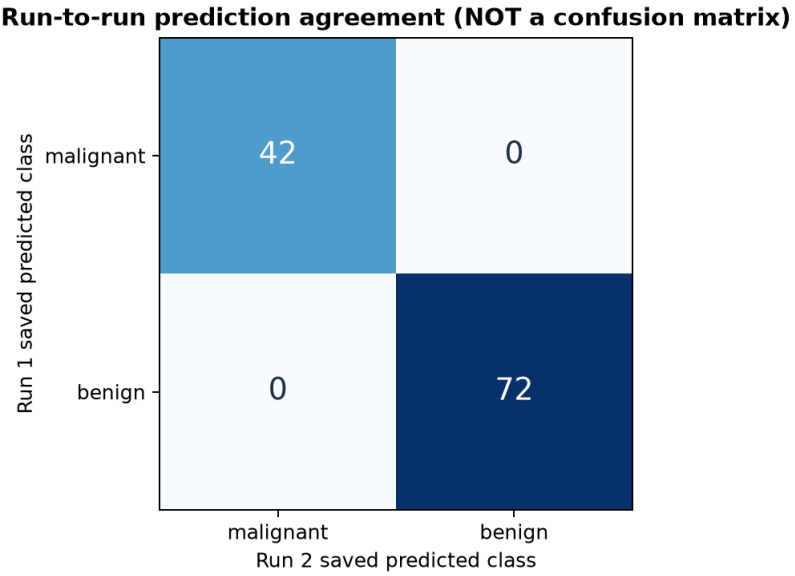


Figure 9. Cross-run agreement between saved prediction classes

The matrix compares Run 1 predictions with Run 2 predictions using aligned record IDs; it is **not a confusion matrix against ground-truth test labels**.

Evidence limitation: The saved prediction CSVs lack true test labels and the positive-class probability for every record. Accordingly, this report does not show a ground-truth confusion matrix, per-threshold ROC curve or precision-recall curve. Those graphs would require saving the original test labels and probabilities during a separately documented evaluation run.

Evidence and Artifact Traceability

Experimental source files: outputs/run_1_dataset_inspection.json; outputs/run_1_metrics.json; outputs/run_2_metrics.json; outputs/reproducibility_comparison.json; outputs/run_1_predictions.csv; outputs/run_2_predictions.csv; models/model.joblib. **Figure reproducibility:** python generate_evidence_plots.py. **Report regeneration:** python generate_report.py.